# Mobility-Aware Collaborative Task Offloading for Parallel Tasks in Vehicular Edge Computing

Jiaxin Du, Jinfan Zhang, Guangjie Han, *Fellow, IEEE*, Mengmeng Wang, Guojiang Shen, Zhi Liu, and Xiangjie Kong, *Senior Member, IEEE*

***Abstract*—The rapid advancement of Internet of Vehicles technology has led to massive growth in vehicular data generation, imposing strict computational demands for latency-sensitive applications. By offloading computational tasks to Road-side Units (RSUs), Vehicular Edge Computing (VEC) offers an efficient solution for those latency-sensitive applications. However, current task offloading schemes generally ignore the time-varying topology caused by vehicle mobility, which poses a risk of task interruption. Moreover, existing task offloading models primarily focus on serial tasks processing and fail to adequately account for the relationships among parallel tasks, leading to inefficient resource utilization and potential latency accumulation in multi-task VEC scenarios. To this end, we propose a mobility-aware Collaborative Task Offloading scheme for Parallel tasks (CoTOP) in VEC. Integrated with vehicle mobility detection, a collaborative task offloading model is designed based on deep reinforcement learning, achieving effective coordination among RSUs to reduce task processing latency. Additionally, a task prioritization algorithm is incorporated to optimize resource allocation. Experimental results show that CoTOP significantly outperforms existing schemes in terms of task processing latency, energy consumption and completion ratio.**

***Index Terms*—Vehicular edge computing, deep reinforcement learning, task offloading, vehicle mobility.**

## I. Introduction

In recent years, the Internet of Vehicles (IoV) has become a critical component of smart cities with the rapid advancement of the Internet of Things and intelligent transportation technologies [1]. This technological evolution has led to dramatically increasing demands for real-time data processing in vehicular environments [2]. However, due to power constraints and limited hardware resources, vehicular computing units often struggle to handle computational tasks independently - particularly for computation-intensive and latency-sensitive applications, such as autonomous driving [3] and route planning [4]. Vehicular Edge Computing (VEC) has emerged as an promising solution to overcome these limitations. By enabling dynamic task offloading to neighboring vehicles or roadside units (RSUs) [5], VEC effectively alleviates the limitations of computational resources while significantly reducing task processing latency [6].

The task offloading frameworks in VEC typically adopt the vehicle-to-RSU paradigm, where vehicles offload computational tasks to a single RSU within wireless communication range [7], [8], [9]. While this paradigm demonstrates satisfactory performance in scenarios with low vehicle density and modest computational demands, it encounters significant limitations in complex dynamic VEC environments [10]. First, vehicles usually traverse multiple RSU service zones during task execution due to limited RSU coverage areas. The high-speed vehicles may exit the serving RSU's communication range before completing task transmission and result retrieval, leading to task interruption or failure [11]. Furthermore, when vehicles generate substantial computing tasks, offloading complete workloads directly to a single RSU inevitably causes task queue congestion, resulting in excessive processing delays [7]. Additionally, the vehicles frequently produce multiple parallel tasks (e.g., local path planning and lane-changing decisions) with varying priorities and resource requirements. Such task characteristics necessitate an efficient collaborative offloading framework to enable parallel task processing [12].

Collaborative task offloading frameworks enhance computational efficiency in VEC by decomposing complex vehicular tasks into smaller subtasks and dynamically offloading them across multiple RSUs [13]. This offloading strategy is optimized based on varying network conditions and the spatial distribution of RSU resources, enabling effective integration of distributed computation and communication capabilities for efficient parallel task execution. Such cooperative approaches demonstrate superior adaptability to the highly dynamic VEC environment compared to conventional offloading schemes, significantly improving both task completion reliability. Recent advances in collaborative offloading frameworks have yielded promising results [14]. For instance, several studies employ Deep Reinforcement Learning (DRL) to dynamically adjust offloading decisions based on some critical factors such as vehicle location,

Received 25 April 2025; revised 28 October 2025; accepted 9 November 2025. Date of publication 12 November 2025; date of current version 6 March 2026. The work was supported in part by the National Natural Science Foundation of China under Grant 62402442, Grant 62073295, and Grant 62476247, in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LQ24F020038, in part by the Open Fund of State Key Laboratory of Acoustics and Marine Information under Grant SKLA202501, and in part by the Zhejiang Province Basic Public Welfare Research Project under Grant LTGG24F020009. Recommended for acceptance by H. Tabassum. *(Corresponding author: Xiangjie Kong.)*

Jiaxin Du, Jinfan Zhang, Mengmeng Wang, Guojiang Shen, Zhi Liu, and Xiangjie Kong are with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou 310014, China (e-mail: dujiaxin@zjut.edu.cn; zhangjinfan@zjut.edu.cn; wangmengmeng@zjut.edu.cn; gjshen1975@zjut.edu.cn; lzhi@zjut.edu.cn; xjkong@ieee.org).

Guangjie Han is with the Key Laboratory of Maritime Intelligent Network Information Technology, Ministry of Education, Hohai University, Changzhou 213022, China, and also with the State Key Laboratory of Acoustics and Marine Information, Chinese Academy of Sciences, Beijing 100190, China (e-mail: hanguangjie@gmail.com).

Digital Object Identifier 10.1109/TMC.2025.3631820

resource availability, and task requirements [14], [15], [16], [17]. Through iterative interaction with the VEC environment, the learning agent develops context-aware offloading decision models across multiple RSUs in dynamic VEC scenarios.

While collaborative task offloading presents a promising solution for complex VEC systems, its practical deployment faces dual challenges. On one hand, the inherent mobility of vehicles imposes fundamental constraints on the cooperation of multiple RSUs [18]. As vehicles traverse RSU coverage areas, the available time window for maintaining stable connections becomes severely limited. Under such dynamic connectivity conditions, computational results generated by RSUs may fail to reach moving vehicles when task execution cycles exceed vehicular dwell time, introducing risks of task failure. Moreover, connection disruptions trigger redundant computations, increasing network overhead, and degrading resource utilization efficiency of RSUs. On the other hand, RSUs must simultaneously handle heterogeneous tasks from multiple vehicles, which exhibit significant variations in computational scale, latency sensitivity, and processing requirements. The first-come-first-served scheduling strategies lack dynamic priority assessment, making them prone to task queue congestion during traffic peaks. This congestion propagates cascading service quality degradation throughout the network, particularly for the time-sensitive tasks.

To address the aforementioned challenges, we propose a mobility-aware Collaborative Task Offloading scheme for Parallel task processing (CoTOP). First, we develop a Graph Attention Network (GAT)-based vehicular mobility detection model to dynamically capture spatiotemporal correlations in vehicle interactions. By analyzing historical trajectory data, this model estimates vehicles' dwell time within the current RSU coverage area, thereby facilitating parallel task scheduling and collaborative offloading while reducing processing delays. Subsequently, we design a task prioritization algorithm that accounts for both vehicle dwell time and task characteristics. Through dynamic evaluation of tasks, the prioritization algorithm institutes differentiated priority levels, which optimizes the allocation of RSU resources while ensuring their efficient utilization. Finally, we formulate the collaborative offloading problem as a joint optimization objective aiming to minimize both task delay and energy consumption. The DRL algorithm is adopted to continuously perceive the dynamic changes in task attributes and vehicle mobility, and achieve optimal collaborative task offloading decisions through online policy updates in dynamic VEC environments. The main contributions of this work are as follows:

- To the best of our knowledge, this is the first work formulating a collaborative task offloading problem integrating vehicle mobility awareness and parallel task computation in VEC environments. This problem is modeled as a joint optimization to simultaneously minimize average task execution delay and energy consumption.
- We propose a vehicle mobility detection model to estimate dwell time at RSU coverage area, enabling collaborative offloading and parallel task processing. Building upon the mobility detection model, we design a task prioritization algorithm to optimize VEC resource allocation, and mitigate the risk of vehicle task offloading failure.
- We develop a DRL-based collaborative task offloading decision model to solve the joint optimization problem. The task offloading strategy is adaptively optimized by dynamically perceiving variations in vehicle positions and task attributes, improving the task completion ratio within complex VEC environments.

The rest of the paper is organized as follows: Section II introduces related work on task offloading. Section III presents the system model and problem description. Section IV provides a detailed explanation of the proposed CoTOP method. Section V discusses the experimental results and evaluations. Section VII concludes the paper. Finally, Section VI discusses directions for future research.

## II. Related Works

In this section, we categorize existing task offloading schemes in VEC into two categories: one relies on a single RSU to handle offloaded tasks independently, while the other employs cooperation among multiple RSUs to complete offloading tasks.

### A. Single-RSU Task Offloading

In the situation of single-RSU task offloading, some studies [19], [20], [21] consider vehicle mobility but restrict tasks to be executed only locally on the vehicle or at its designated RSU. Li et al. [19] address scenarios where vehicles are mobile and tasks arrive dynamically, proposing a two-layer hybrid system that supports both local and edge computing. They formulate a comprehensive latency optimization problem and solve it using a deep reinforcement learning algorithm. Hou et al. [20] established a multi-modal cooperative computing offloading model that integrates Vehicle-to-Vehicle, Vehicle-to-RSU (V2R), Vehicle-to-MBSs and Vehicle-to-Cloud based on the mobility of vehicles. They optimized the task offloading latency by developing an offline RL methods with augmented data. Guo et al. [21] build a queuing model for task flows, jointly considering latency, energy consumption, and user quality of service requirements, and propose a computing function solved by the DDPG algorithm with a priority experience replay mechanism to determine the optimal offloading decision. Fofana et al. [22] consider issues such as network fluctuations and develop a model suitable for IoV task offloading, taking into account both latency and communication overhead; they model the offloading strategy as a four-stage game combined with deep reinforcement learning to find optimal solutions. Although single-RSU task offloading can enhance the efficiency of vehicular edge computing systems to some degree, the high-speed mobility of vehicles and the constantly changing road environment cause dynamic fluctuations in the communication and computing resources needed by RSUs. Consequently, single-RSU approaches often struggle to adapt to these fluctuations. They are unable to flexibly adjust resource scheduling and optimization based on vehicle trajectories and task requirements, resulting in lower offloading efficiency and even offloading failures.

### B. Multi-RSU Cooperative Task Offloading

With the rapid development of the IoV, the high mobility of vehicles and the limited communication range of RSUs have prompted researchers to explore cooperative approaches among

TABLE I
COMPARISON OF TASK OFFLOADING RESEARCH

| Scheme | Objectives | | Collaboration | Mobility | Priority | Parallel task |
|---|---|---|---|---|---|---|
| | Latency | Energy | | | | |
| RRTD3 [14] PDDPG [21] | ✓ | ✓ | ✓ | | | |
| DODQ [25] OTPPS [26] | ✓ | | | | | ✓ |
| MSA3C [17] MBDNN [24] GWCN [27] | ✓ | ✓ | ✓ | ✓ | | |
| TORAS [23] A-TARFD [6] POSDR [5] | ✓ | ✓ | | ✓ | | |
| BPSO [28] | ✓ | ✓ | | | | ✓ |
| MCMA [16] | ✓ | | ✓ | ✓ | | |
| TODDPG [19] | ✓ | | | ✓ | | |
| ORAD [20] | ✓ | | | | | |
| AIoT [29] | ✓ | | | | ✓ | |
| MDRCO [30] | | ✓ | | | | |
| MESON [11] | ✓ | ✓ | ✓ | ✓ | ✓ | |
| **Ours** | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

multiple RSUs to enhance the reliability and efficiency of task offloading. Fan et al. [14] consider RSU load conditions in IoV and propose several cooperative strategies among RSUs to boost offloading efficiency. Zhang et al. [16] propose a multi-agent deep reinforcement learning-based collaborative optimization framework for task offloading in multi-edge computing scenarios. By dynamically coordinating edge node resource allocation via an attention mechanism, their approach formulates task latency and system energy consumption as a joint optimization objective, significantly enhancing resource utilization in dynamic environments. Zhao et al. [17] construct a VEC environment in which tasks that cannot be handled by a local RSU can be forwarded to nearby RSUs or even the cloud. Their aim is to minimize offloading delay, and they adopt an asynchronous advantage actor-critic (A3C)-based solution. Fan et al. [23] propose a heterogeneous network architecture for IoV, treating task latency and energy consumption as a joint optimization problem. They develop a branch-and-bound algorithm enhanced with linear relaxation to optimize resource allocation.

Although multi-RSU cooperative task offloading offers substantial advantages in addressing the high mobility of vehicles and the resource limitations of individual RSUs, certain challenges remain in current research: (1) High collaboration overhead, as existing work often overlooks the transmission overhead involved in cooperative task offloading; (2) Low collaboration efficiency, because few studies thoroughly investigate the timing and conditions under which cooperative task offloading should be initiated.

To clarify our research findings, we compare them with previous studies in Table I. The information presented in the table highlights the contributions of our work.

## III. System Model and Problem Formulation

In this section, we first describe the system model in the VEC environment, including the task model, communication model, computation model, and energy consumption model. Then, we define the task offloading problem as a joint optimization problem that considers both task execution delay and energy consumption.

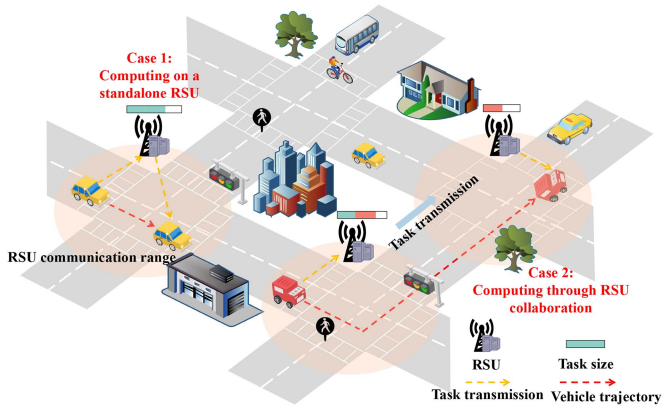


Fig. 1. The system architecture of collaborative task offloading in VEC-based vehicular network.

### A. Task Model

We consider a vehicular network environment consisting of multiple vehicles and RSUs deployed along a roadway. The overall system architecture is depicted in the Fig. 1. In this system, vehicles and RSUs function as two distinct types of computing nodes, each characterized by differing computational resources and processing capabilities. The RSUs, uniformly distributed along the road, are represented by the set $R = \{R_1, R_2, \ldots, R_M\}$, while the moving vehicles are represented by the set $V = \{V_1, V_2, \ldots, V_N\}$. The system operates in discrete time slots, collectively denoted by the set $\mathcal{T} = \{1, 2, \ldots, T\}$. During each time slot $t \in T$, a vehicle $V_n$ generates a set of $I$ tasks, represented as $S = \{s_{n,1}, s_{n,2}, \ldots, s_{n,I}\}$. Each RSU $R_m$ needs to handle a task $s_{n,i}(t)$ in each time slot, which is denoted as $\{\rho_{n,i}(t), \varphi_{n,i}(t), d_{n,i}(t)\}$, where $\rho_{n,i}(t)$ represents task size generated by vehicle $V_n$ in task $i$ during the time slot $t$, $\varphi_{n,i}(t)$ indicates the number of CPU resources required for the task $i$, and $d_{n,i}(t)$ signifies the maximum delay time for the task $i$. For each task, during task offloading, if the vehicle remains within the communication range of the current RSU, the task is computed on a standalone RSU (Case 1); if it enters another RSU's range, the task is computed through RSU collaboration (Case 2). In the Fig. 1, the green portion in the task size represents the amount of task data computed locally at the current RSU, while the red portion indicates the task data transmitted to another RSU for collaborative computation.

### B. Communication Model

In our scenario, RSUs can not only communicate with vehicles within their coverage area but also interact with neighboring RSUs to collaboratively perform task offloading. Therefore, the calculation of the transmission rate can be divided into two categories: V2R transmission and RSU-to-RSU (R2R) transmission.

*1) V2R Transmission:* At time slot $t$, the transmission rate $w_{n,m}^{V2R}$ between vehicle $V_n$ and RSU $R_m$ is expressed

as:

$$w_{n,m}^{V2R}(t) = B^{V2R}\log_2\left(1+\frac{P^V K}{\omega {D_{n,m}}^{\sigma}}\right), \tag{1}$$

where $B^{V2R}$ represents the network bandwidth, $P^V$ denotes the transmission power of the vehicle, $K$ represents the fixed loss, $\omega$ denotes the noise power, $D_{n,m}$ represents the distance between the vehicle $V_n$ and the RSU $R_m$, and $\sigma$ represents the path loss factor.

*2) R2R Transmission:* Similar to (1), the transmission rate between RSU $R_m$ and RSU $R'_m$ is given by:

$$w_{m,m'}^{R2R}(t) = B^{R2R}\log_2\left(1+\frac{P^R K}{\omega {D_{m,m'}}^{\sigma}}\right), \tag{2}$$

Unlike the dynamic positioning of vehicles, which causes fluctuations in the transmission rate over time, RSUs are stationary. As a result, the distance between RSUs remains unchanged, ensuring a constant transmission rate.

### C. Computation Model

*1) Computing on a Standalone RSU:* At the end of all time slots, if the vehicle remains within the coverage of the current RSU, the scenario is referred to as RSU standalone computation. This process consists of three stages: the vehicle offloads the task to the RSU; the RSU completes the computation of the task; the RSU returns the computation results to the vehicle. Consequently, three distinct delays arise: $T_{n,m,i}^{up}(t)$ for task offloading, $T_{n,m,i}^{pro}(t)$ for task processing, and $T_{m,n,i}^{b}(t)$ for transmitting the computation results. Given that the data size of the computation results is significantly smaller than that of the task data, the delay associated with transmitting the results is negligible. From the transmission model above, the transmission rate between the vehicle $V_n$ and the RSU $R_m$ can be used to calculate $T_{n,m,i}^{up}(t)$ as:

$$T_{n,m,i}^{up}(t) = \frac{\rho_{n,i}(t)}{w_{n,m}^{V2R}(t)}. \tag{3}$$

Let $F_m^{RSU}$ represent the computing ability of RSU, thus, for the task $s_{n,i}(t)$ generated by vehicle $V_n$ at time $t$, the task processing delay $T_{n,m,i}^{pro}(t)$ can be expressed as:

$$T_{n,m,i}^{pro}(t) = \frac{\varphi_{n,i}(t)}{F_m^{RSU}}. \tag{4}$$

Considering that upon arrival at the RSU, the current task may need to wait due to other tasks already queued for processing, the task cannot begin execution immediately. Consequently, the waiting delay must be taken into account. The waiting delay $T_{m,i}^{wait}(t)$ experienced by task $s_{n,i}(t)$ on RSU $R_m$ can be calculated using the follow formula:

$$T_{m,i}^{wait}(t) = \frac{N_m^{queue}(t)}{F_m^{RSU}}. \tag{5}$$

Because the delay in returning the calculation results can be ignored, at time slot $t$, the total computation delay $T_{n,m,i}^{total}(t)$ for task $s_{n,i}(t)$ generated by vehicle $V_n$ on a single RSU $R_m$ can be expressed as:

$$T_{n,m,i}^{total}(t) = T_{n,m,i}^{up}(t) + T_{n,m,i}^{pro}(t) + T_{m,i}^{wait}(t). \tag{6}$$

*2) Computing Through RSU Collaboration:* When a serving RSU detects during time slot $t$ that a vehicle will exit its communication range, inter-RSU collaborative computation becomes necessary to ensure completion of the ongoing task. This collaborative process unfolds in four stages:

- *Initially task offloading stage:* The vehicle initially offloads the task to the current RSU.
- *Inter-RSU transmit stage:* The RSU wirelessly transmits partially processed data and remaining tasks to the next RSU connecting to the vehicle.
- *Successive computation stage:* The next RSU resumes and completes the computation of the remaining task.
- *Result delivery stage:* The next RSU transmits the completed computation results to the vehicle.

Unlike traditional cooperative task offloading methods, our approach leverages a mobility detection model to detect when a vehicle will leave the communication range of an RSU. This allows us to determine the amount of task data that can be executed within the current RSU and the portion that cannot be completed before the vehicle moves out of range. By identifying this division of task data, we can process the second and third stages in parallel, significantly improving the efficiency of the task offloading process.

As delineated in Fig. 2, three critical temporal parameters govern this process: $t_0$ denotes the transmission delay for uploading the task to the current RSU. $t_1$ represents the time calculated by the mobility detection model when the vehicle leaves the RSU's coverage area. $t_2$ refers to the transmission delay for offloading the remaining task to the next RSU, while $t_3$ denotes the computational delay for processing the remaining portion of the task, and $t_4$ represents the transmission delay for returning the computed results from the RSU to the vehicle. The task offloading process in this scenario shares similarities with the first case in terms of calculating the upload delay and the waiting delay on the RSU. However, it differs in how the computational delay is handled. By accurately detecting the vehicle's mobility and segmenting the task based on the time constraints $t_1$, our approach ensures that computational resources are effectively utilized, minimizing unnecessary waiting times and improving overall system efficiency. This parallelized strategy not only reduces delay but also enhances the reliability of task completion in highly dynamic vehicular environments. Based on the time $t_1$, the amount of task data $\rho_{n,m,i}^{rest}(t)$ that can not be completed can be calculated using the following formula:

$$\varphi_{n,m,i}^{rest}(t) = \varphi_{n,i}(t) - t_1 \times F_m^{RSU}. \tag{7}$$

Once the remaining unprocessed task size is determined, data transmission can commence between RSU $R_m$ and RSU $R'_m$, and the transmission delay $T_{m,m',i}^{ts}(t)$ between RSU $R_m$ and RSU $R_{m'}$ calculated using the following formula: can be calculated using the following formula:

$$T_{m,m',i}^{ts}(t) = \frac{\rho_{n,m,i}(t)}{w_{m,m'}^{R2R}(t)}, \tag{8}$$

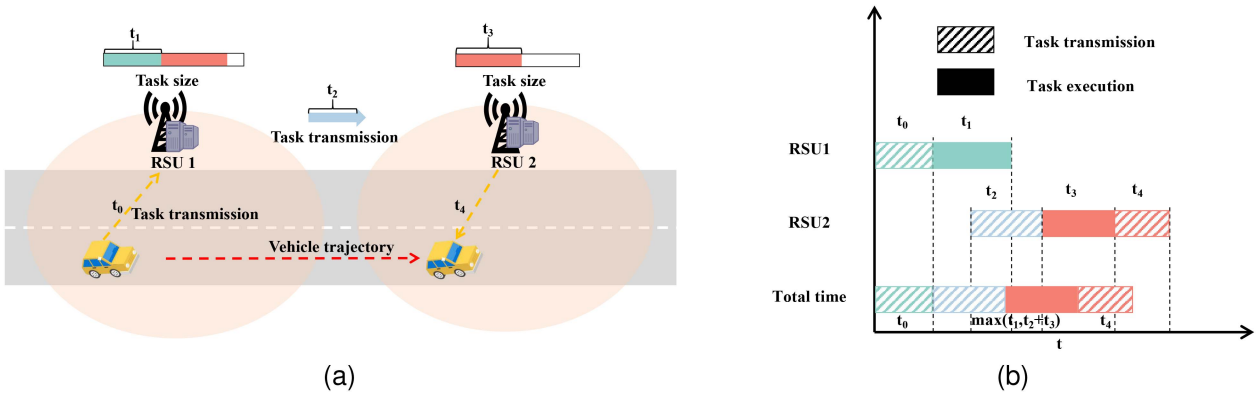


Fig. 2. Illustration of task computing through RSU collaboration in VEC.

and the time $T^{pro\_rest}_{n,m',i}(t)$ required to complete the remaining calculations is :

$$T^{pro\_rest}_{n,m',i}(t) = \frac{\varphi^{rest}_{n,m,i}(t)}{F^{RSU}_{m'}}. \tag{9}$$

As illustrated in Fig. 2, $t_2$ corresponds to the inter-RSU transmission delay, while $t_3$ characterizes the computational delay associated with the remaining calculation. Thus, the final computational delay $T^{pro}_{n,m',i}(t) = \max(t_1, t_2 + t_3)$. The calculation of other delays follows a similar approach to that of standalone RSU computation. Therefore, the total delay for task $s_{n,i}(t)$ generated by vehicle $V_n$ at time $t$, which is collaboratively processed by RSU $R_m$ and RSU $R_{m'}$, can be expressed as:

$$T^{total}_{n,m,m',i}(t) = T^{up}_{n,m,i}(t) + T^{pro}_{n,m',i}(t) + T^{wait}_{m',i}(t). \tag{10}$$

### D. Energy Consumption Model

We divide the energy consumption model into two components: communication energy consumption and computation energy consumption. Communication energy consumption refers to the energy required for data transmission between vehicles and RSUs, as well as between RSUs. Computation energy consumption represents the energy required to complete the computation of a task. The computation energy consumption is calculated using the following formula:

$$E^{pro}_{i}(t) = \begin{cases} T^{pro}_{n,m,i}(t) E^{RSU}_{m}, & x_{i,r} = 0 \\ (T^{pro\_rest}_{n,m',i}(t) + t_1) E^{RSU}_{m}, & x_{i,r} \neq 0 \end{cases}. \tag{11}$$

where $E^{RSU}_{m}$ represents the computation power consumption of the RSU, and $x_{i,r}$ denotes the decision on collaborative offloading, $x_{i,r} = 0$ indicates no collaboration is initiated, and $x_{i,r} \neq 0$ indicates collaborative offloading is performed. The calculation of transmission energy consumption is influenced by the decision of whether the task is completed collaboratively. The calculation formulas are as follows:

$$E^{ts}_{i}(t) = \begin{cases} T^{up}_{n,m,i}(t) E^{V2R}, & x_{i,r} = 0 \\ T^{up}_{n,m,i}(t) E^{V2R} + T^{ts}_{m,m',i}(t) E^{R2R}, & x_{i,r} \neq 0 \end{cases}. \tag{12}$$

Here, $E^{V2R}$ represents the communication energy consumption between the vehicle and the RSU, while $E^{R2R}$ represents the communication energy consumption between RSUs. Therefore, the total energy consumption $E^{total}_{i}(t)$ for a task can be defined as the sum of $E^{pro}_{i}(t)$ and $E^{ts}_{i}(t)$.

### E. Problem Formulation

In each time slot $t$, each vehicle generates a task and offloads it to the RSU for execution. Our objective is to minimize the average execution delay of all tasks while taking into account the load conditions of the RSUs. We define a unity function $U(t)$ as the average execution delay and average energy consumption of the tasks generated by the vehicles. The calculation of $U(t)$ is as follows:

$$U_m(t) = \frac{\sum_{i=1}^{I} \left(\sigma T^{\text{total}}_{i}(t) + (1-\sigma) E^{\text{total}}_{i}(t)\right)}{I}, \tag{13}$$

where $T^{total}_{i}(t)$ and $E^{total}_{i}(t)$ represent the delay and energy consumption for computing task $s_{n,i}(t)$, and $\sigma \in [0,1]$ is a tunable weighting parameter that quantifies the relative importance of conflicting objectives. This parameter enables flexible prioritization adjustments across different operational scenarios. To systematically balance the trade-off between computation delay and energy efficiency, we formulate the collaborative task offloading problem in the VEC environment as a constrained multi-objective optimization problem as follows:

$$\min \sum_{m=1}^{M} \sum_{t=1}^{T} U_m(t) \tag{14}$$

$$\text{s.t.} \quad C1: v(t) \leq v_{max}, t > 0 \tag{14a}$$

$$C2: T^{total}_{i} \leq d_i, \forall i \in I \tag{14b}$$

$$C3: N^{queue} \leq C_{RSU} \tag{14c}$$

$$C4: \sum_{i=1}^{I} E^{total}_{i} \leq E^{max}, \forall i \in I \tag{14d}$$

$$C5: \sum_{t=1}^{T} \varphi_i(t) \leq \varphi_i, \forall i \in I \tag{14e}$$

Constraints $C1$ and $C2$ ensure that the speed of each vehicle does not exceed the maximum allowable speed and that

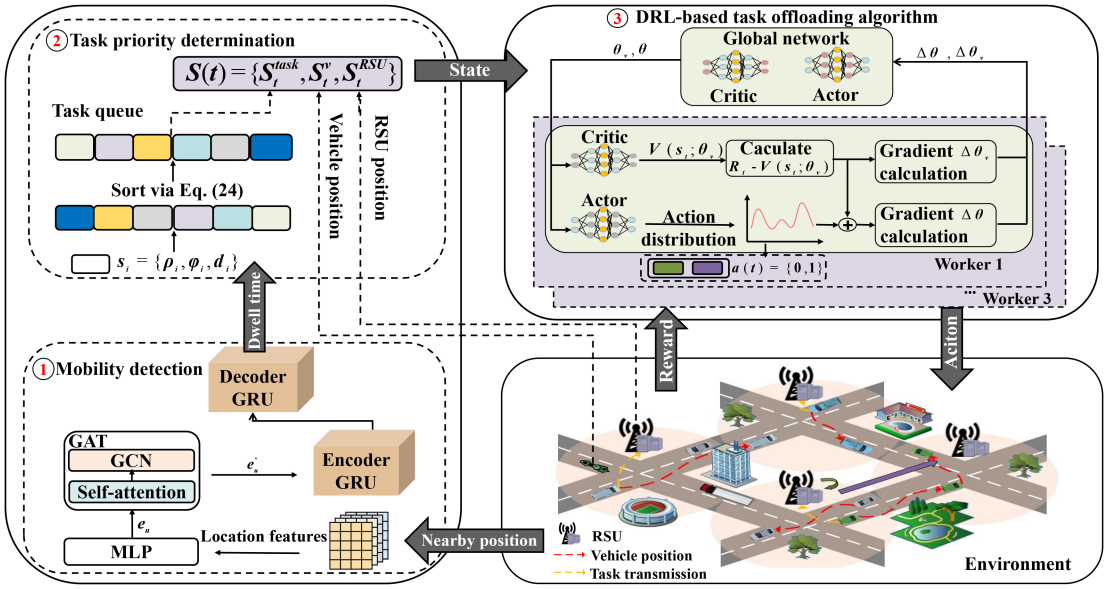


Fig. 3. Framework of the proposed CoTOP scheme.

the execution time of each task remains within its maximum tolerable limit, respectively. Constraint $C3$ guarantees that the computational workload assigned to each RSU does not surpass its maximum computational capacity and $C_{RSU}$ represents the maximum computational capacity of a RSU. Constraint $C4$ ensures that the energy consumption generated by task computation stays within the maximum allowable energy limit to maintain system efficiency. Finally, constraint $C5$ ensures that the amount of data processed for each task at any given time does not exceed the task's maximum data size, ensuring proper resource allocation and task feasibility.

## IV. Mobility-Aware Collaborative Task Offloading

### A. Overview

The framework of the proposed CoTOP scheme is illustrated in the Fig. 3. First, we propose a mobility detection model capable of estimating vehicles' dwell time within the current RSU coverage area, enabling parallel task scheduling and collaborative offloading to reduce processing latency. Building upon the dwell time estimation, we design a differentiated priority algorithm that ensures timely processing of mission-critical tasks while optimizing resource allocation. Finally, we construct a DRL-based task offloading algorithm that dynamically adapts to vehicular network conditions through continuous environment interaction, achieving intelligent resource coordination between vehicles and RSUs. The subsequent sections will detail the design principles and implementation specifics of each module.

### B. Mobility Detection Model Based on GAT

In vehicular networks, the dynamic mobility of vehicles often results in them moving beyond the communication range of their current RSU, causing potential disruptions in communication. To address this issue, collaboration among RSUs is necessary to ensure the completion of task offloading, rather than abandoning the vehicle's tasks. To tackle this challenge, we propose a vehicle mobility detection model capable of detecting vehicle positions within specific time intervals, thereby optimizing task offloading decisions. As an essential auxiliary tool for task offloading in vehicular networks, trajectory detection plays a vital role in enhancing the reliability of offloading decisions. Our proposed mobility detection model transforms vehicle data into a graph-structured representation, which is then processed using a GAT to extract meaningful spatial and temporal features. An attention mechanism is incorporated to prioritize critical features, further improving the model's accuracy. The model utilizes an encoder-decoder framework, enabling efficient and precise vehicle position detection within dynamic environments. The specific parameter settings of the model are shown in the Table II.

TABLE II
GAT, GRU and Overall Model Training Hyperparameter Configuration

| module | parameters | value |
|---|---|---|
| GAT | Input size | 32 |
| | Output size | 32 |
| | Attention heads | [4,1] |
| | Hidden units | 16 |
| | Activation | LeakyReLU |
| GRU | Input size | 64 |
| | Hidden units | 2 |
| | Number of layers | 2 |
| | Dropout rate | 0.5 |
| Overall model training | Epochs | 500 |
| | Batch size | 64 |
| | Learning rate | 0.01 |
| | Optimizer | Adam |

The mobility detection model takes the historical positional coordinates $(x, y)$ of both vehicles and surrounding objects

(e.g., pedestrians, bicycles) as input. First, we use a two-layer Multilayer Perceptron (MLP) to expand the dimension of the input. The expanded output is obtained using the following formula:

$$e_u = MLP(x_u, y_u), x_u \in \mathbb{R}^{16}, e_u \in \mathbb{R}^{32}. \tag{15}$$

Then, the output $e_u$ obtained from the two-layer MLP is used as the input for the GAT. In GAT, each node computes attention scores with its neighboring nodes based on their features, enabling the model to dynamically learn the importance of relationships between nodes. The attention scores are calculated using the following formula:

$$\alpha_{u,v} = \frac{\exp(\text{LeakyRelu}(a^T[We_u||We_v]))}{\sum_{k\in N(u)} \exp(\text{LeakyRelu}(a^T[We_u||We_k]))}, \tag{16}$$

where $\alpha_{u,v}$ is the attention weights between node $u$ at time $i$ and node $v$ at time $j$, while $W \in \mathbb{R}^{F'\times F}$ is a shared trainable matrix, $F'$ is the output feature dimension and $F$ is the input feature dimension. Additionally, $||$ denotes the concatenation operation of two matrices, and $N(u)$ represents the set of neighboring nodes of node $u$ at the same time. Furthermore, $\alpha$ is a weight vector used to assign importance to specific features.

To effectively capture dynamically changing topological relationships in vehicular networks, we designed a dual-layer graph attention mechanism. After computing attention scores, this mechanism processes spatiotemporal feature inputs from preceding modules, which contain vehicle and environmental information. In the first GAT layer, a multi-head attention mechanism operates four independent feature analysis channels in parallel, enabling vehicles to simultaneously perceive multi-dimensional critical features of their surroundings. The process of hierarchical aggregation of multi head attention mechanism is as follows:

$$e_{u'} = \sum_{k=1}^{4} \sum_{v\in N(u)} \alpha_{u,v} W_k e_u, \tag{17}$$

where $W_k \in \mathbb{R}^{32\times 32}$ is the weight matrix. Subsequently, single-head attention aggregation compresses the concatenated high-dimensional features through dimensionality reduction, which preserves core relational information while significantly reducing computational load. So the final output obtained through GAT is:

$$e_{u''} = W' \cdot MEAN\left(e'_{u,1} + e'_{u,2} + e'_{u,3} + e'_{u,4}\right). \tag{18}$$

The processing through GAT enables each vehicle's feature representation to incorporate not only its own historical position information but also dynamically integrate critical features from neighboring vehicles, effectively capturing the spatial dependencies among them. This capability is particularly vital for vehicle position detection models, as they fully exploit the interaction information between vehicles to better model dynamic position changes in complex traffic scenarios. Additionally, GAT enhances the model's sensitivity to significant positional changes by assigning higher weights to critical neighbors. This improves the accuracy of position detection and provides a more reliable foundation for optimizing task offloading decisions.

The position of a vehicle is influenced not only by spatial relationships but also by its dynamic positional changes over time. As a variant of recurrent neural networks, the Gated Recurrent Unit (GRU) can effectively capture long-term dependencies in sequential data. After processing through MLP and GAT layers, we obtain an input sequence $e = \{e_1, e_2, \ldots, e_T\}$ comprising $T$ time steps, where $e_T = (x_T, y_T)$ represents the vehicle's position at time $T$. The encoder sequentially processes this input sequence through a series of GRU units, ultimately generating the hidden state $h_T$:

$$h_T^{encoder} = GRU(e_T, h_{T-1}), \tag{19}$$

which encapsulates the temporal information of the entire historical trajectory. The decoder, implemented as another GRU, utilizes $h_T$ as contextual information to detect the vehicle's future position over several time steps. At each step, the decoder calculates a detected position and feeds the current hidden state into the next time step's input. Through this iterative process, the decoder constructs the trajectory points for the vehicle's future movements. The generation of vehicle positions follows the formula below:

$$h_{T+k} = GRU\left(h_{T+k-1}, h_T^{encoder}\right), \tag{20}$$

$$\hat{x}_{T+k} = W \times h_{T+k} + b. \tag{21}$$

The detected coordinates are evaluated by calculating the mean squared error between them and the true coordinates, which serves as the loss function. This loss is used to optimize the network parameters, enabling the model to progressively improve the precision of subsequent location determinations. The formula for the loss calculation is as follows:

$$Loss = \frac{1}{N}\sum_{k=1}^{K}\left[(\hat{x}_{T+k} - x_{T+k})^2 + (\hat{y}_{T+k} - y_{T+k})^2\right]. \tag{22}$$

### C. Task Priority Determination

This section elaborates on the task priority determination algorithm proposed in this study. The algorithm systematically incorporates the critical influence of task prioritization on system efficiency, with the objective of optimizing holistic system performance through scientifically-justified task sequencing. Specifically, the priority determination framework operates under rigorously defined evaluation criteria, ensuring prioritized processing of mission-critical tasks to enhance both efficiency and reliability in task offloading processes:

- Tasks with stricter time constraints have higher priority.
- Tasks with larger data volumes are assigned higher priority.
- Tasks from vehicles with longer dwell times are given higher priority.

Accordingly, the priority of task $s_{n,i}$ is defined as:

$$P_i = \alpha e^{-\frac{1}{T^{stay}}} + \beta\frac{\rho_{n,i}}{d_{n,i}}, \tag{23}$$

where $T^{stay}$ represents the dwell time of the vehicle within the RSU's coverage area, indicating that the longer the dwell time, the closer $e$ is to 1, resulting in a higher priority. $\alpha$ and

$\beta$ are weighting coefficients, and $\alpha + \beta = 1$. $\rho_{n,i}$ represents the task size, while $d_{n,i}$ represents the task's maximum delay. This priority determination algorithm ensures tasks exhibiting both substantial data volume and stringent latency constraints are assigned higher processing priority, thereby enhancing resource utilization efficiency in VEC systems.

### D. DRL-Based Task Offloading Algorithm

*1) MDP Construct:* To optimize the collaboration strategy among offloading tasks, we designed a decision-making algorithm based on DRL. This algorithm enables autonomous learning to select the optimal task offloading collaboration strategy based on the states of vehicles and RSUs, thereby enhancing the task completion ratio while reducing delay and energy consumption. Reinforcement learning is a learning methodology that maximizes cumulative rewards through interaction with the environment. In the context of collaborative decision-making for tasks, we model it as a Markov Decision Process (MDP), which provides a mathematical framework for sequential decision-making. Specifically, the MDP models the interaction between an agent and its environment, where the agent continuously learns the optimal policy by exploring states, performing actions, receiving rewards, and undergoing state transitions to maximize the expected long-term cumulative return. Collectively, these components define the optimization objective of the task offloading collaboration strategy.

*State space:* The state $s_t$ represents the environmental information of the VEC at time slot $t$. Since the entire IoV environment includes vehicles, RSUs, and tasks generated by vehicles, we define the state space at time slot $t$ as:

$$s(t) = \left\{ s_t^v, s_t^{task}, s_t^{RSU} \right\}, \tag{24}$$

where $s_t^v = \{x_t, y_t\}$, represents the position of the vehicles, $s_t^{task}$ describes the relevant details of tasks, and $s_t^{RSU}$ indicates the amount of task data on the RSU.

*Action space:* The action space $a(t)$ defines the decision-making process at each time slot $t$, determining whether the RSU currently associated with a vehicle should collaborate to complete the task and, if so, selecting the appropriate collaboration target. Specifically, $a(t) = \{0, 1, \ldots, M\}$, where 0 indicates no collaboration, and $M$ denotes the ID of the RSU selected for collaboration.

*Reward function:* Since the objective of the system is to minimize the total execution delay and energy consumption of all tasks, the reward at time slot $t$ is defined as the negative value of the total delay and energy consumption of all tasks up to time $t$. When defining the task model, a maximum tolerable time is set for each task. During the delay calculation process, if the total delay exceeds the task's maximum tolerable time or the distance of the vehicle from the current RSU surpasses the coverage range of two RSUs, the reward is assigned a fixed value $Z$, which can be regarded as a penalty. This negative feedback helps the model learn more reasonable strategies, improving the overall quality of task offloading decisions. Therefore, the reward function is defined as follows:

$$r(t) = \begin{cases} -(\epsilon T^{total}(t) + (1-\epsilon) E^{total}(t)), T^{total}(t) \le d_{n,i} \\ -Z, T^{total}(t) > d_{n,i} \end{cases}. \tag{25}$$

where $\epsilon \in [0, 1]$, denotes the regularization parameter governing the trade-off between computational delay and energy consumption in task processing,

*2) The Training Process of DRL-Based Algorithm:* In the VEC environment, task offloading must account for dynamic scenarios such as vehicle mobility, RSU resource allocation, and uncertainties in task attributes (e.g., data volume and delay requirements). Traditional task offloading strategies often fail to satisfy real-time performance and optimization demands. To address these challenges, we employ a training framework based on the A3C algorithm.

The training process is illustrated in the Fig. 3. In the previous sections, we have modeled the entire VEC-based task collaboration system as a MDP, where the agent interacts with the environment and makes collaborative task decisions iteratively. A3C integrates policy optimization and value function estimation, employing asynchronous parallel worker threads to accelerate the training process. Through the combined effect of the policy network (actor) and the value network (critic), A3C optimizes collaborative decisions efficiently. This makes A3C particularly well-suited for achieving effective collaborative decision-making in complex VEC environments. By exploring the environment in parallel across multiple threads and sharing experiences with the global network, the A3C algorithm significantly reduces training time and enhances policy stability.

At time slot $t$, each thread copies the parameters from the global network. For each thread, there is a separate actor network and critic network. The actor outputs the probability distribution of actions $\pi = (a_t | s_t; \theta')$ based on the current state $s_t$, where $\theta'$ represents the parameters of the actor. The goal of the actor is to select an action $a_t$ that maximizes future rewards. After taking the action $a_t$, the agent receives a corresponding reward and the state transitions to $a_{t+1}$. The task of the critic network is to estimate the state value function $V(s_t; \theta_{v'})$, which represents the expected return the vehicle can achieve under the current policy given the state $s_t$. The parameter of the critic network is denoted by $\theta_{v'}$.

The objective during the training process for each thread is to minimize the loss values of both the actor network and the critic network. The loss function of the actor network is as follows:

$$L_\pi(\theta') = -\log_\pi (a_t | s_t; \theta') (R_t - V(s_t; \theta_{v'})), \tag{26}$$

where $R_t$ represents the cumulative reward, and its calculation formula is defined as:

$$R_t = \sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k}; \theta_{v'}), \tag{27}$$

where $\gamma$ is the discount factor, used to determine the weight of the future rewards, where $\gamma \in [0,1]$. The loss function of the critic network is as follows:

$$L_V(\theta_{v'}) = (R_t - V(s_t | \theta_{v'}))^2. \tag{28}$$

**Algorithm 1.** CoTOP Task Offloading.

1: Initialize global parameters: actor network weights $\theta$, critic network weights $\theta_v$ and global counter $T = 0$;
2: **for** each parallel thread **do**
3: Initialize workers' parameters $\theta' \leftarrow \theta$ and $\theta'_v \leftarrow \theta_v$;
4: **while** $T < T_{\max}$ **do** :
5: Reset gradients: $\Delta\theta \leftarrow 0$, $\Delta\theta_v \leftarrow 0$;
6: Synchronize parameters: $\theta' \leftarrow \theta$, $\theta'_v \leftarrow \theta_v$;
7: Initialize state $s(0)$ and the reward $r(0) = 0$;
8: **for** each time slot $t$ **do**
9: Detect vehicle positions for $t + 1$;
10: Compute task priority based on (23);
11: Sort tasks by priority;
12: Select an action $a_t$ using $\pi(a_t|s_t;\theta')$;
13: Execute $a(t)$ to obtain reward $r(t)$;
14: Update the state $s_t \leftarrow s_{t+1}$
15: Update the critic by minimizing the $L_V(\theta_{v'})$;
16: Accumulate gradients:
17: $\Delta\theta \leftarrow \Delta\theta + \nabla_{\theta'} \log \pi(a_t|s_t;\theta') \cdot A$;
18: $\Delta\theta_v \leftarrow \Delta\theta_v + \nabla_{\theta'_v}(R - V(s_t;\theta'_v))^2$;
19: **end for**
20: Update global parameters $\theta$ and $\theta_v$;
21: Increment global counter $T \leftarrow T + 1$;
22: **end while**
23: **end for**

After completing training, each thread transmits its gradient updates to the global network. The global network updates its parameters accordingly to ensure continuous optimization. At the start of the next training iteration, all threads copy the updated global parameters and continue training based on the new states. This iterative process refines the agent, enabling it to make optimal collaborative decisions and effectively maximize long-term rewards.

The specific training process of the CoTOP task offloading scheme based on the A3C algorithm is outlined in Algorithm 1. The collaborative decision module of each RSU is equipped with our proposed strategy. First, global parameters are initialized, including the weights of the actor and critic networks as well as a global counter (Line 1). At the beginning of each thread, the global network parameters are synchronized, and thread-specific actor and critic network weights are initialized (Line 3). During each time slot in a training iteration, gradient values are reset to zero (Line 5), global network parameters are synchronized through shared memory, and the actor/critic network weights are initialized with the latest parameter values (Line 6). Subsequently, the environment state and reward values are initialized (Line 7). The mobility detection model detects the vehicle's position in the next time slot, thereby adjusting the feasibility of tasks (Line 9). Simultaneously, tasks are prioritized based on factors such as data size, maximum tolerable delay, and the vehicle's dwell time (Line 10). Subsequently, the actor network outputs the probability distribution of actions based on the current state and selects the optimal action to execute task collaboration (Lines 12–14). The critic network estimates the advantage value by calculating the difference between the current state value function and cumulative reward, which is then used to optimize both the actor and critic networks (Line 15). At the end of each time slot, the accumulated gradients are applied to update the global network parameters, completing one training iteration (Lines 16-18). Through parallel training across multiple threads, the CoTOP task offloading scheme effectively optimizes vehicle task offloading strategies while significantly improving task completion ratio and reducing execution delays and energy consumption.

### *E. Analysis of CoTOP Algorithm*

*1) Computational Complexity of CoTOP Algorithm:* Here we present the computational complexity of GAT. The time complexity of GAT primarily depends on three components: the feature transformation, the attention coefficient calculation, and neighbor aggregation. First, for the feature transformation, each node $u$ requires $\mathcal{O}(F \cdot F')$ Multiply-Add operations, hence the overall computation complexity is $\mathcal{O}(N \cdot F \cdot F')$, where $N$ is the total number of nodes.

For attention coefficient calculation, each node and its neighbors perform a dot product operation with dimensionality $2F'$ (as specified in (16)). Thus, the overall computation complexity of this step is $\mathcal{O}(|E| \cdot 2F')$, where $|E|$ denotes the total number of directed edges in the graph. Prior to neighbor aggregation, attention coefficients undergo normalization. Although this process involves exponential operations, each edge computes the $exp()$ function only once, effectively requiring neighbor summation per node. Consequently, the normalization complexity is $\mathcal{O}(\sum_{u=1}^{N} |N_u|) = \mathcal{O}(|E|)$, where $N_u$ is the neighbor node of $u$. The total computational complexity for this phase is therefore $\mathcal{O}(|E|) + \mathcal{O}(|E| \cdot 2F') = \mathcal{O}(|E|)$.

In neighbor aggregation, a weighted summation of features from each node $u$ and its neighbors is performed to derive new representations. Given each node has $d_u$ neighbors, the computational complexity for this operation is $\mathcal{O}(\sum_{u=1}^{N} F' \cdot d_u + F') = \mathcal{O}(\sum_{u=1}^{N} F' \cdot d_u) + \mathcal{O}(N \cdot F') = \mathcal{O}(F' \cdot |E|) + \mathcal{O}(N \cdot F')$, because $|E| \gg N$, the complexity is $\mathcal{O}(|E| \cdot F')$.

Add up the complexity of all the above steps and ignore the smaller of the low order term and the added term. The complexity of GAT is

$$\mathcal{O}\left(N \cdot F \cdot F' + |E| \cdot F'\right). \tag{29}$$

The Actor (Critic) network structure consists of $J$ ($L$) DNN layers. In the Actor network, the number of neural units in the $j^{th}$ layer is $\eta_j$ ($\mu_j$). Therefore, the total computational complexity for the Actor network is $\mathcal{O}(\sum_{j=0}^{J} \eta_j \eta_{j+1})$. Similarly, the total computational complexity for the Critic network is $\mathcal{O}(\sum_{l=0}^{L} \mu_l \mu_{l+1})$. In our proposed CoTOP, there are three FC layers for the Actor and Critic network.

The computational complexity of the A3C with three FC layers is given by

$$\mathcal{O}\left(H\left(\sum_{j=0}^{J} \eta_j \eta_{j+1} + \sum_{l=0}^{L} \mu_l \mu_{l+1}\right)\right)$$

$$= \mathcal{O}\left(H\left(\sum_{j=0}^{3}\eta_j\eta_{j+1} + \sum_{l=0}^{3}\mu_l\mu_{l+1}\right)\right), \tag{30}$$

where $H$ is the training steps in the training phase.

The computational complexity of CoTOP method that combines GAT and A3C is

$$\mathcal{O}\left(N \cdot F \cdot F' + |E| \cdot F'\right) + \mathcal{O}\left(H\left(\sum_{j=0}^{3}\eta_j\eta_{j+1} + \sum_{l=0}^{3}\mu_l\mu_{l+1}\right)\right). \tag{31}$$

In sparse graph structures, the number of nodes in GAT and the feature dimensions are relatively small. Through quantitative comparison, the contribution of the computational complexity term of GAT to the total system complexity is significantly lower than that of the reinforcement learning policy. Therefore, the final computational complexity of the CoTOP algorithm can be effectively approximated as:

$$\mathcal{O}\left(H\left(\sum_{j=0}^{3}\eta_j\eta_{j+1} + \sum_{l=0}^{3}\mu_l\mu_{l+1}\right)\right). \tag{32}$$

*Remark 1.* The computational complexity of reinforcement learning policy optimization depends on three key factors: the training step size $H$, the number of fully-connected (FC) layers, and the number of neural units $\eta_j(\mu_j)$ in each FC layer. It should be highlighted that the GAT computational term may be neglected under sparse graph conditions, whereas when both node population and edge count substantially increase, the computational complexity attributable to GAT cannot be arbitrarily omitted.

*2) Convergence Analysis of CoTOP Algorithm:* The convergence analysis of the CoTOP algorithm is established upon the classical Actor-Critic framework. The analysis of the critic network is based on the theoretical framework and assumptions proposed by Shen et al. [31]. In their work, the authors introduced a series of key assumptions (Assumptions 2–4) and rigorously derived the convergence results of the Actor-Critic algorithm. Since the optimization structure and update mechanism of the proposed CoTOP algorithm are consistent with this framework, the corresponding convergence analysis is also applicable. We first provide the following theorem on the convergence of the critic network update:

*Theorem 1 (Critic convergence).* Based on Assumptions 2–4 in [31]. Consider Algorithm 1 with i.i.d. sampling and $\hat{V}_\omega(s) = \phi(s)^T\omega$. By selecting the step sizes $\alpha = K^{-\frac{1}{2}}$ and $\beta = K^{-\frac{1}{2}}$, the update error of the critic parameters in the CoTOP algorithm satisfies the following bound:

$$\frac{1}{K}\sum_{k=1}^{K}\mathbb{E}\left\|\boldsymbol{\omega}_k - \boldsymbol{\omega}^*_{\theta_k}\right\|_2^2 = \mathcal{O}\left(\frac{K_0^2}{K^{\frac{1}{2}}}\right) + \mathcal{O}\left(\frac{K_0}{K^{\frac{1}{2}}}\right) + \mathcal{O}\left(\frac{1}{K^{\frac{1}{2}}}\right). \tag{33}$$

The result shows that the critic network's parameter estimation error converges at $\mathcal{O}(K^{-\frac{1}{2}})$, with convergence affected by the maximum delay $K_0$.

*Theorem 2 (Actor convergence).* After establishing the convergence of the critic network, we further analyze the convergence of the actor network. Under the same assumptions as Theorem 1, and selecting step sizes $\alpha = K^{-\frac{1}{2}}$ and $\beta = K^{-\frac{1}{2}}$, the expected average of the squared gradient norm of the convergence function for the actor network $\nabla J_\lambda(\theta_k)$ satisfies the following equality:

$$\frac{1}{K}\sum_{k=1}^{K}\mathbb{E}\left\|\nabla J_\lambda(\theta_k)\right\|_2^2 = \mathcal{O}\left(\frac{1}{K^{\frac{1}{2}}}\right) + \mathcal{O}\left(\frac{K_0^2}{K^{\frac{1}{2}}}\right) + \mathcal{O}\left(\frac{K_0}{K^{\frac{1}{2}}}\right) + \mathcal{O}\left(\epsilon_{\text{app}}\right). \tag{34}$$

Furthermore, if the maximum delay $K_0$ and the number of worker nodes $N$ satisfy $K_0 = \Theta(N) = \mathcal{O}(K^{\frac{1}{2}})$, the convergence rate can be optimized to:

$$\frac{1}{K}\sum_{k=1}^{K}\mathbb{E}\left\|\nabla J_\lambda\left(\theta_k\right)\right\|_2^2 = \mathcal{O}\left(K^{-\frac{1}{2}}\right) + \mathcal{O}\left(\epsilon_{\text{app}}\right). \tag{35}$$

Here, the constants implicit in the $\mathcal{O}(.)$ term are independent of the number of worker nodes $N$ the delay $K_0$.

### F. Theoretical Analysis of Resource Utilization

To provide a clear theoretical guarantee for the effective utilization of RSU resources under the proposed optimization strategy, we introduce a queueing theory model to analyze the task processing of RSUs. This analysis helps quantify the relationship between RSU utilization, task delay, and queue stability, and, in particular, offers theoretical support for the reliability and practicality of the method under heavy-load conditions.

Each RSU is modeled as a single-server queue, where the arrival process represents tasks offloaded from vehicles within its coverage area, and the service process corresponds to the RSU's computational processing capability. Let $\lambda_m$ denotes the average task arrival rate to RSU $R_m$, which is calculated by the formula:

$$\lambda_m = N_m \cdot \bar{r} \cdot \bar{p}, \tag{36}$$

where $N_m$ indicates the average number of vehicles in the RSU coverage area, $\bar{r}$ denotes the average task generation rate per vehicle, and $\bar{p}$ represents the average offloading rate to $R_m$. Let $\mu_m$ denote the service rate of $R_m$, computed as the ratio between its computational capacity $F_m^{RSU}$ and the average computation demand per task $\bar{\varphi}$.

The RSU utilization factor $\chi_m$ is defined as:

$$\chi_m = \frac{\lambda_m}{\mu_m}. \tag{37}$$

For an $M/M/1$ queue, the stability condition $\chi_m < 1$ ensures that the average queue length and waiting time remain finite.

In the following, we analyze the situation when the RSU is under high load. In the experimental scenario, there are at most 30 vehicles at the same time, so the maximum value of $N_m$ is 30. Each time slot $t$, each vehicle generates a computational task, i.e., the task generation rate $\bar{r}$ is 1. The proportion $\bar{p}$ of tasks that are offloaded to the RSU on average is determined

TABLE III
SIMULATION PARAMETERS

| Parameter | Value |
|---|---|
| Number of Vehicles | [10, 30] |
| Number of RSUs | 6 |
| Vehicle Speed | [30, 40] m/s |
| RSU Computational Capacity | [1, 4] Gcycles/s |
| Number of Tasks | [20, 40] |
| Task Size | [2, 5] MB |
| Task Tolerance Time | [20, 30] s |
| RSU Communication Range | 400 m |
| Vehicle Transmission Power | 10 dBm |
| RSU Transmission Power | 50 dBm |
| Vehicle Bandwidth | [20, 100] Mbps |
| Bandwidth Between RSUs | 50 MHz |
| Noise Power | 0.001 dBm |
| Fixed Loss | 30 dB |
| Path Loss Exponent | 2 |

by the collaborative offloading policy, and it can be up to a maximum of 1. Based on this, the average task arrival rate $\lambda_m$ is at a maximum of 30 task/s. In time slot $t$, the maximum CPU resource requirement of each generated task $\varphi_i(t)$ is 10 Mcycles, so the average computation demand per task $\bar{\varphi}$ is a maximum of 10 Mcycles. At the same time, the computational capacity of each $R_m$ is a minimum of 1GHz. Based on this, the minimum service rate of $R_m$ $\mu_m$ is 100. Therefore, the utilization factor of a single RSU $\chi_m$ will not be more than 0.3, which can maintain the stability of the system.

## V. EXPERIMENTS AND RESULTS

This section evaluates the performance of the proposed CoTOP through a systematic comparison with state-of-the-art baseline methods under varying road conditions. We first describe the experimental scenario and evaluation metrics, followed by an introduction of the comparative methods. Then, we provide an objective assessment of CoTOP by comparing it with existing approaches. Finally, we validate the effectiveness of the CoTOP under changing road conditions in real-world scene.

### A. Experimental Scenario and Evaluation Metrics

Our experiments combine real-world trajectory data with simulation-based traffic modeling to evaluate our task offloading strategy. First, our GAT-based mobility detection model is trained using the ApolloScape trajectory dataset [32], a high-precision dataset collected from complex urban road scenarios in China. The dataset contains spatio-temporal motion trajectories and category labels, enabling our model to accurately detect vehicle dwell times within RSU coverage areas. Simulation experiments were conducted on a Windows 10 64-bit operating system using Python 3.8 and PyTorch 2.4.1. The environment settings for the experiments are shown in the Table III. A simulation map was downloaded from OpenStreetMap and used to generate traffic data flows through the Simulation of Urban Mobility (SUMO). This map is a real urban scene in Hangzhou Province, China. The size of the map is 200 m $\times$ 200 m. In this environment, 10 to 30 vehicles were deployed, each randomly generating initial position coordinates and speeds. Six RSUs were evenly distributed along the roadside, with all RSUs having the same communication range and computational capabilities. Next, we present the evaluation metrics employed in our experiments.

- *Average Delay*: Average delay represents the average time required to complete all tasks in the VEC system. It is an important metric for reflecting the efficiency of task offloading.
- *Average Energy Consumption*: Average energy consumption represents the average energy required for the transmission and processing of all tasks in the VEC system.
- *Completion Ratio*: Completion ratio represents the ratio of the number of tasks completed within their tolerable time to the total number of tasks in the VEC system. It is a critical metric for evaluating the reliability and stability of the offloading strategy.

### B. Approaches for Comparison

To verify the effectiveness of the proposed CoTOP scheme, we conducted a comparative analysis with several commonly used baseline methods. The detailed descriptions of these baseline methods are as follows:

- **QRMP-DQN** [33]: The Quantile Regression Multi-Pass Deep Q-Network (QRMP-DQN) is an algorithm based on DQN that jointly optimizes task offloading and resource allocation to minimize the average energy consumption of the system. DQN is a reinforcement learning algorithm that combines deep learning with Q-learning.
- **DDQN** [34]: Double Deep Q-Network (DDQN) is an enhanced DRL algorithm based on the DQN framework. Its core mechanism employs deep neural networks to achieve nonlinear approximation of the optimal action-value function, thereby addressing the overestimation bias inherent in the conventional DQN framework.
- **Local**: This is an offloading method without any collaboration between RSUs. In this approach, each task generated by a vehicle is offloaded to an RSU and can only be computed by the current RSU without being forwarded to other RSUs.
- **Greedy**: In this method, each RSU makes collaborative decisions based on a certain optimization objective by selecting the most suitable task at the current moment. This method does not consider the long-term impact of collaborative decisions on the system and only selects locally optimal solutions.

### C. Hyperparameter Analysis

We conducted experimental studies to investigate the impact of the learning rate on the convergence performance of our proposed CoTOP scheme. As shown in Fig. 4, the convergence performance of CoTOP improves significantly as the learning rate decreases, manifested by an increase in reward values, reduced post-convergence fluctuations, and enhanced stability. Considering both convergence behavior and reward performance, we ultimately set the learning rate to 0.0002.

Under the scenario of 10 vehicles and 25 tasks, we systematically investigated the parameters $\alpha$ and $\beta$ in (23). As shown in the

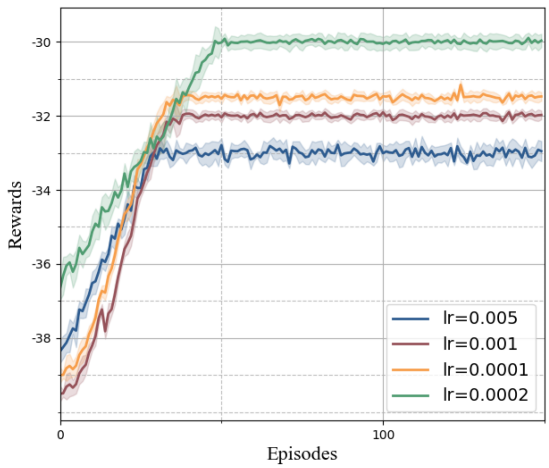


Fig. 4. Convergence of CoTOP with different learning rates.

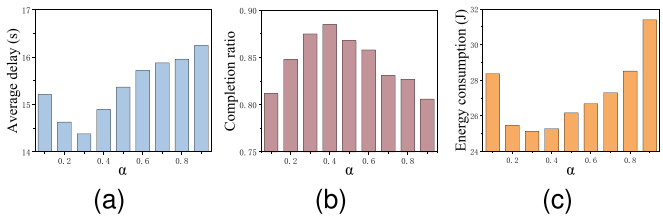


Fig. 5. The impact of hyperparameter $\alpha$.

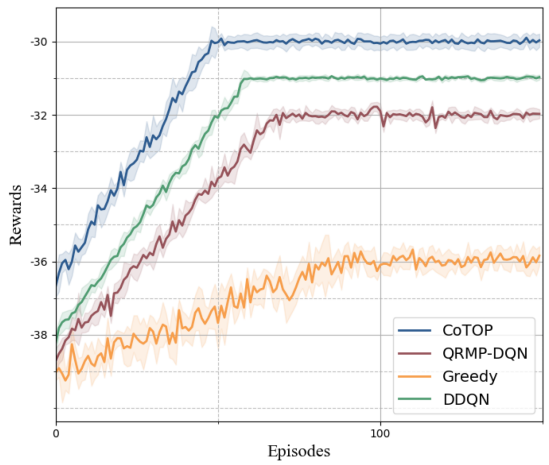


Fig. 6. Convergence of different task offloading methods.

Fig. 5, the average delay initially decreases then increases with $\alpha$, reaching the minimum value at $\alpha = 0.3$ , while significantly rising to 16s as $\alpha$ increases from 0.2 to 0.8. The task completion ratio peaks at 0.88 when $\alpha = 0.4$ . Energy consumption remains stable around 25J near $\alpha = 0.3$, but shows a marked increase with higher $\alpha$ values. Comprehensive evaluation suggests $\alpha = 0.3$ achieves optimal balanced performance for task offloading.

### D. Performance Comparison

We first analyze the convergence of the CoTOP. As shown in Fig. 6, it can be observed that all three methods converge to a stable reward value after a relatively small number of iterations.

TABLE IV
COMPARISON OF AVERAGE DELAY AMONG DIFFERENT METHODS UNDER VARYING NUMBERS OF TASKS

| Task Numbers | CoTOP | QRMP-DQN | DDQN | Local | Greedy |
|---|---|---|---|---|---|
| 20 | **13.9** | 14.7 | 14.2 | 18.7 | 16.4 |
| 25 | **14.3** | 15.2 | 15.3 | 19.0 | 16.5 |
| 30 | **14.9** | 15.9 | 15.8 | 19.6 | 16.8 |
| 35 | **15.2** | 16.3 | 16.2 | 19.7 | 16.9 |
| 40 | **15.5** | 16.7 | 16.7 | 20.2 | 17.2 |

TABLE V
COMPARISON OF COMPLETION RATIO AMONG DIFFERENT METHODS UNDER VARYING NUMBERS OF TASKS

| Task Numbers | CoTOP | QRMP-DQN | DDQN | Local | Greedy |
|---|---|---|---|---|---|
| 20 | **0.91** | 0.86 | 0.89 | 0.52 | 0.51 |
| 25 | **0.86** | 0.84 | 0.85 | 0.46 | 0.48 |
| 30 | **0.84** | 0.82 | 0.84 | 0.45 | 0.46 |
| 35 | **0.82** | 0.78 | 0.81 | 0.37 | 0.45 |
| 40 | **0.78** | 0.77 | 0.79 | 0.38 | 0.43 |

However, the reward value achieved by the Greedy method is relatively low. Although the QRMP-DQN method achieves a higher reward value, its reward exhibits significant fluctuations in the later stages of training. While DDQN demonstrates relatively stable convergence, its convergence speed is slower compared to the CoTOP. In contrast, the proposed CoTOP demonstrates the best performance in terms of reward stability and maximum value, indicating its superior robustness in dynamic and complex environment.

In vehicular network environments, the number of tasks has a significant impact on task offloading efficiency. Tables IV and V present the average delay and completion ratio under different task numbers. As shown in the Table IV, the average delay of all methods increases with the growing number of tasks. Notably, the DDQN method demonstrates superior performance compared to traditional DQN methods, benefiting from its double Q-learning mechanism that mitigates value overestimation. However, DDQN still underperforms relative to the CoTOP, as the actor-critic architecture in A3C framework enables better adaptation to high-dimensional state spaces in dynamic environments. The CoTOP scheme consistently achieves the lowest delay with minimal fluctuation as task numbers increase. This stability stems from CoTOP's efficient task allocation strategy and dynamic policy adjustments that effectively handle environmental complexity. The QRMP-DQN method exhibits slightly higher delay than CoTOP, primarily due to its limited adaptability in high-dimensional action spaces. Both Greedy and Local method demonstrate substantially higher delay than CoTOP and QRMP-DQN, with the Local approach showing the worst performance due to its exclusive reliance on RSU computational resources.

The task completion ratio, as illustrated in Table V, align closely with the delay analysis, with CoTOP consistently outperforming other methods by achieving the highest task completion ratio. QRMP-DQN and DDQN follow with moderate yet reliable performance, while Greedy and Local methods exhibit notably

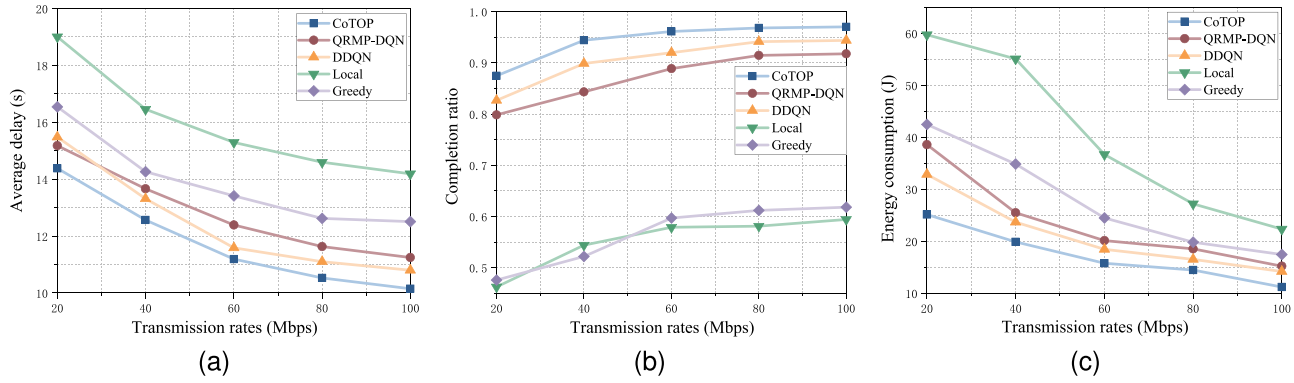


Fig. 7. Performance comparison under different transmission rates.

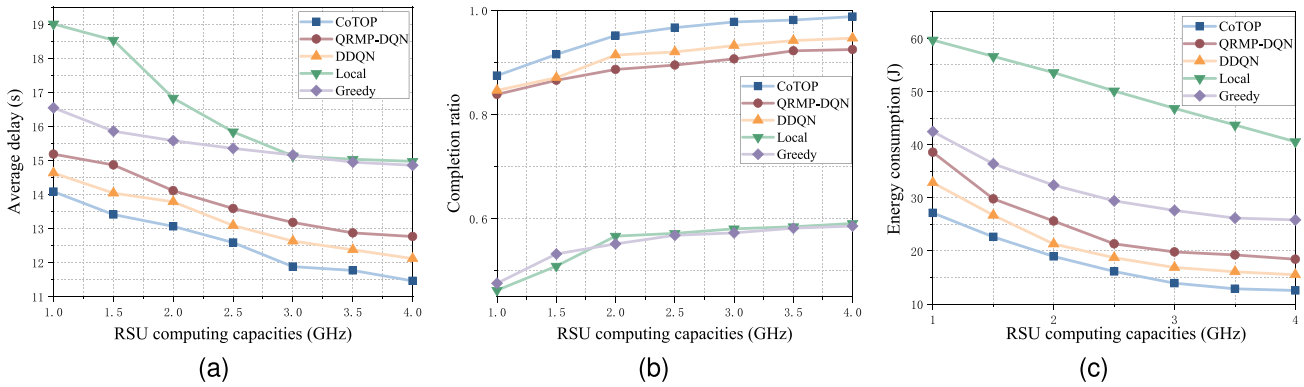


Fig. 8. Performance comparison under different RSU computing capacities.

lower completion ratio, underscoring their inherent limitations in handling complex task allocation scenarios.

In addition to the impact of task numbers on task offloading efficiency, network conditions in vehicular environments also play a significant role. Fig. 7 illustrates the impact of transmission rate on task offloading efficiency. From Fig. 7(a), it can be observed that as the transmission rate increases, the average delay of all methods decreases significantly. Among the methods, the CoTOP achieves the lowest average delay due to its efficient task allocation strategy. The DDQN method demonstrates competitive performance, achieving lower delay than both Greedy and Local methods, though slightly higher than CoTOP and QRMP-DQN. In contrast, the Local method exhibits the highest average delay, as it does not perform collaborative offloading, leading to a lower task completion ratio. The Greedy method reduces queuing delays by offloading tasks to the RSU with the least load. However, due to the mobility of vehicles, task offloading reliability is compromised, causing task failures and higher delays compared to CoTOP, QRMP-DQN, and DDQN. The CoTOP method outperforms QRMP-DQN and DDQN in task allocation and adjustment capabilities, resulting in lower average delays.

Fig. 7(b) shows the task completion ratio under different transmission rates. The CoTOP, QRMP-DQN, and DDQN methods exhibit higher completion ratio due to their exploration capabilities, which allow them to learn better offloading strategies. DDQN's dual-network architecture enables stable Q-value estimation, contributing to its reliable task completion performance across varying transmission rates. In contrast, the Greedy and Local methods lack exploration capabilities, leading to relatively lower task completion ratio. Finally, Fig. 7(c) reveals that as the transmission rate increases, the average energy consumption of all algorithms shows a decreasing trend. This is because higher transmission rates reduce the task transmission time, thereby lowering the corresponding transmission energy consumption. The DDQN and QRMP-DQN methods significantly outperform the Greedy and Local approaches in terms of energy efficiency, while CoTOP consistently demonstrates the lowest energy consumption.

We conducted comparative experiments to analyze the impact of RSU computational capacity on offloading efficiency. As shown in Fig. 8, the average delay of all methods decreases with increasing RSU computing capacities. The Local method consistently exhibits the highest delay, though its downward trend remains relatively pronounced. The Greedy method demonstrates slightly lower delays than the Local approach but still underperforms compared to QRMP-DQN. The DDQN method shows intermediate characteristics in delay performance, positioned between the Greedy and QRMP-DQN methods. This occurs because the Greedy method fails to adapt to dynamic action space variations, leading to suboptimal offloading decisions under high-load conditions. In contrast, the CoTOP method maintains the lowest delay consistently through its optimization capabilities in dynamic task offloading scenarios. Regarding task

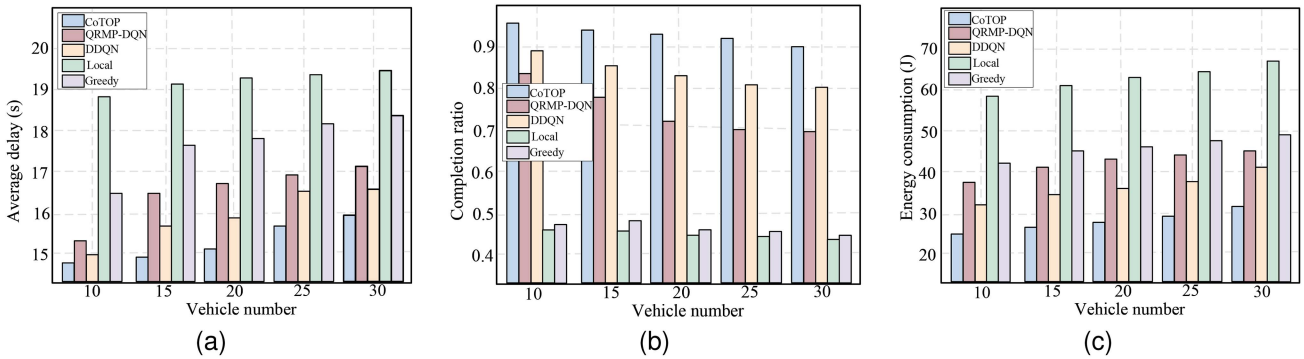


Fig. 9. Performance comparison under different numbers of vehicles.

TABLE VI
RESULTS OF ABLATION EXPERIMENT

| Metric | CoTOP | w/o MD | w/o TP | w/o CO |
|---|---|---|---|---|
| Average delay | **13.1** | 15.5 | 14.5 | 16.4 |
| Completion ratio | **0.87** | 0.68 | 0.82 | 0.55 |
| Energy consumption | **25.14** | 15.32 | 33.52 | 49.15 |

completion and energy efficiency metrics, the DDQN method outperforms conventional rule-based approaches but remains slightly inferior to enhanced deep reinforcement learning methods. While QRMP-DQN can partially utilize RSU resources in complex environments, its limitations in high-dimensional action spaces constrain overall performance, resulting in higher average delays than CoTOP. The experimental results on completion ratio and energy consumption further validate the effectiveness of CoTOP in optimizing task offloading efficiency.

Fig. 9 illustrates the impact of the number of vehicles on task offloading efficiency in vehicular network environments. As the number of vehicles increases, the average task completion delay rises for all methods, while the task completion ratio gradually decreases. Additionally, the CoTOP, DDQN and QRMP-DQN methods exhibit significant fluctuations in both average delay and task completion ratio, whereas the Greedy and Local methods show relatively minor variations. The primary reason for this phenomenon is that CoTOP, DDQN and QRMP-DQN methods frequently adjust their task allocation strategies in response to dynamic task distributions and the growing number of vehicles, leading to greater performance fluctuations. In contrast, the Greedy and Local methods, which rely on simpler strategies, handle task offloading more consistently but lack the efficiency of more advanced methods. Despite the fluctuations, the CoTOP consistently achieves the lowest average delay and highest task completion ratio, outperforming all other methods.

Finally, we conducted ablation experiments on individual modules in the proposed CoTOP method, where $MD$ denotes the mobility detection module, $TP$ represents the task priority module, and $CO$ indicates the collaborative offloading module. The experimental results are illustrated in the Table VI. As demonstrated, each module plays an important role in the performance of task offloading, such as delay and energy consumption. After removing the mobile detection module, the average delay and energy consumption of completing tasks have significantly increased, while the task completion ratio has significantly decreased, indicating the importance of the mobile detection module for task offloading. After removing the task priority module, the efficiency in task scheduling decreases, resulting in a certain degree of performance degradation. If collaboration is not carried out, the experimental results are most severely affected, indicating that collaborative offloading is crucial for optimizing collaboration efficiency. Through ablation experiments, we have verified the important role of each module in improving task offloading efficiency and resource utilization.

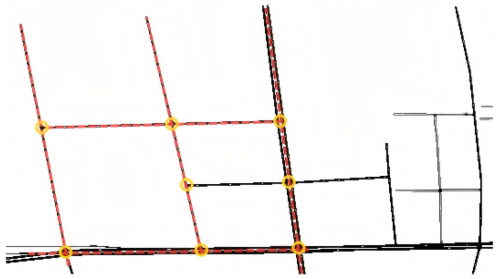

Fig. 10. The simulation map of Hangzhou road network.

### E. Evaluation of CoTOP in Real-World Road Scene

The experiments and result analysis in the above subsections are derived on our synthetic road environment, and in order to verify the practical feasibility of the CoTOP method, we obtained the local road network data of Hangzhou through "OpenStreet Map", and the simulation map is shown in Fig. 10, which contains 5 main roads (marked in red) and 8 intersections (marked in yellow). We used SUMO to conduct simulation experiments, deployed RSUs uniformly at each intersection, and set up more than 100 vehicles to conduct task offloading tests to simulate the urban dense road scenarios.

Under the aforementioned experimental scenario, we calculated the average delay, energy consumption, and completion ratio of all tasks, and the corresponding results are presented in Fig. 11. As illustrated in the Fig. 11, it can be observed that in real road environments, when the number of vehicles exceeds 100, the latency and energy consumption of the Local method increase significantly, while the task completion ratio

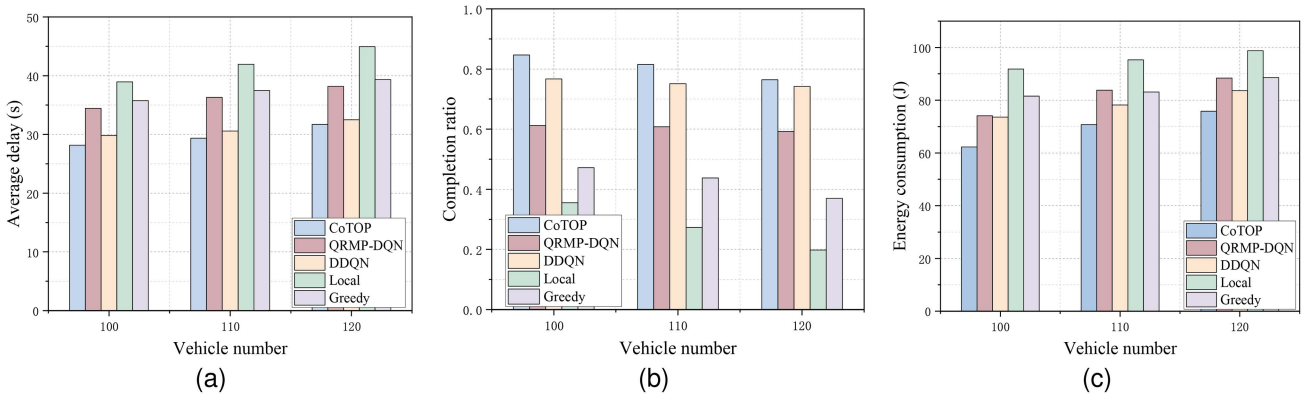


Fig. 11. Performance comparison under different numbers of vehicles on real city roads.

drops sharply. The underlying reason is that once the task scale reaches a certain level, the computing capability of the vehicles themselves can no longer handle such a heavy computational load, resulting in some tasks failing to be completed within the maximum tolerable time. The Greedy method prioritizes offloading tasks to vehicles or RSUs with lighter loads. As a result, when the number of tasks increases, it does not suffer from increased task latency or task failures due to insufficient computing resources. For the other three reinforcement learning algorithms, their ability to dynamically perceive environmental states and update policies in real time enables them to outperform the aforementioned two methods in terms of both latency and energy consumption. Among them, CoTOP consistently achieves the best performance; while DDQN exhibits higher latency and energy consumption compared to CoTOP, it still outperforms the QRMP-DQN algorithm. This is because, compared with DQN, DDQN can effectively mitigate the overestimation of Q-values, thereby achieving better decision-making accuracy and stability.

## VI. Discussion

In this section, we analyze the feasibility and limitations of CoTOP and outline directions for future work.

*Integration feasibility:* We assume that CoTOP is deployed on each RSU, enabling the RSUs to perceive the surrounding environmental state and make optimal task offloading decisions. However, existing VEC systems typically consist of heterogeneous hardware platforms and software protocols, and the computational capacity, storage resources, and communication interfaces of different edge computing nodes can vary significantly, posing challenges for modular integration. In future work, we plan to implement CoTOP as a flexible, microservice-based architecture module that can be adaptively deployed on RSUs, in-vehicle edge nodes, or other MEC servers, depending on the available resources of each edge node.

*Communication model:* When the communication conditions are simple and stable, our current communication model can reduce the complexity of analysis and experiments. However, this model does not fully account for the various interference factors present in real wireless environments. In future work, we plan to incorporate more realistic communication characteristics, such as time-varying channels, path loss, and interference, to more accurately simulate the effects of task transmission latency and data packet loss on system performance. This approach will allow us to evaluate the robustness and adaptability of CoTOP in practical network environments, providing more reliable theoretical and experimental support for its deployment in large-scale vehicular edge computing scenarios.

*Performance trade-offs:* In vehicular edge computing environments, task execution latency remains the most critical performance metric, as many tasks, which include autonomous driving decisions, emergency braking, and collision warnings, have stringent real-time requirements. While low latency is the primary objective, system design must also balance energy consumption and task completion rate. Future research will consider scenario-specific factors, including vehicle density, task types, edge node load, and network conditions, to quantitatively evaluate and optimize energy efficiency and task completion rate while maintaining low latency, thereby improving overall system performance and user experience.

## VII. Conclusion

In this paper, we propose a novel CoTOP scheme that achieves joint optimization of task execution delay and energy consumption in dynamic VEC environments. CoTOP effectively addresses communication instability caused by high-speed vehicle mobility, thereby resolving issues of task interruptions or failures. Additional, by establishing differentiated task priorities, CoTOP effectively mitigates latency increase and energy consumption growth caused by imbalanced resource allocation. CoTOP significantly enhances adaptability to complex and highly dynamic VEC environments, delivering an efficient and reliable solution for real-time services.

## References

[1] Q. Gao et al., "Optimization of models and strategies for computation offloading in the Internet of Vehicles: Efficiency and trust," *IEEE Trans. Mobile Comput.*, vol. 24, no. 4, pp. 3372–3389, Apr. 2025.

[2] S. Shen, G. Shen, Z. Dai, K. Zhang, X. Kong, and J. Li, "Asynchronous federated deep-reinforcement-learning-based dependency task offloading for UAV-assisted vehicular networks," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31561–31574, Oct. 2024.

[3] J. Wang, C. Jiang, K. Zhang, T. Q. S. Quek, Y. Ren, and L. Hanzo, "Vehicular sensing networks in a smart city: Principles, technologies and applications," *IEEE Wireless Commun.*, vol. 25, no. 1, pp. 122–132, Feb. 2018.
[4] J. Yin, W. Rao, Y. Xiao, and K. Tang, "Cooperative path planning with asynchronous multiagent reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 24, no. 6, pp. 5016–5030, Jun. 2025, doi: 10.1109/TMC.2025.3526979.
[5] S. Tian, X. Zhu, B. Feng, Z. Zheng, H. Liu, and Z. Li, "Partial offloading strategy based on deep reinforcement learning in the Internet of Vehicles," *IEEE Trans. Mobile Comput.*, vol. 24, no. 7, pp. 6517–6531, Jul. 2025, doi: 10.1109/TMC.2025.3543976.
[6] Z. Nan, S. Zhou, Y. Jia, and Z. Niu, "Joint task offloading and resource allocation for vehicular edge computing with result feedback delay," *IEEE Trans. Wireless Commun.*, vol. 22, no. 10, pp. 6547–6561, Oct. 2023.
[7] K. Li et al., "Computation offloading in resource-constrained multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 11, pp. 10665–10677, Nov. 2024.
[8] J. Wang, Z. Li, H. Liu, T. Qiu, and H. Luo, "A trust-based computation offloading framework in mobile cloud-edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 6, pp. 5370–5385, Jun. 2025, doi: 10.1109/TMC.2025.3530480.
[9] H. Jiang, J. Cai, Z. Xiao, K. Yang, H. Chen, and J. Liu, "Vehicle-assisted service caching for task offloading in vehicular edge computing," *IEEE Trans. Mobile Comput.*, vol. 24, no. 7, pp. 6688–6700, Jul. 2025, doi: 10.1109/TMC.2025.3545444.
[10] P. Dai, Y. Huang, K. Hu, X. Wu, H. Xing, and Z. Yu, "Meta reinforcement learning for multi-task offloading in vehicular edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 3, pp. 2123–2138, Mar. 2024.
[11] L. Zhao et al., "MESON: A mobility-aware dependent task offloading scheme for urban vehicular edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4259–4272, May 2024.
[12] K. Gu, Z. Liu, and W. Jia, "Location-aware reliable task cooperative-computation scheme under fog computing-based IoVs," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 1, pp. 425–442, Jan. 2025.
[13] Z. Li, C. Yang, X. Huang, W. Zeng, and S. Xie, "CoOR: Collaborative task offloading and service caching replacement for vehicular edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 7, pp. 9676–9681, Jul. 2023.
[14] W. Fan, Y. Zhang, G. Zhou, and Y. Liu, "Deep reinforcement learning-based task offloading for vehicular edge computing with flexible RSU-RSU cooperation," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7712–7725, Jul. 2024.
[15] S. Pratap, P. Dass, and S. Misra, "CoTEV: Trustworthy and cooperative task execution in Internet of Vehicles," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2915–2926, Apr. 2024.
[16] X. Zhang, C. Wang, Y. Zhu, J. Cao, and T. Liu, "Multi-agent deep reinforcement learning with trajectory prediction for task migration-assisted computation offloading," *IEEE Trans. Mobile Comput.*, vol. 24, no. 7, pp. 5839–5856, Jul. 2025, doi: 10.1109/TMC.2025.3539945.
[17] W. Zhao, Y. Cheng, Z. Liu, X. Wu, and N. Kato, "Asynchronous DRL-based multi-hop task offloading in RSU-assisted IoV networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 1, pp. 546–555, Feb. 2025.
[18] X. Chen, J. Cao, Y. Sahni, M. Zhang, Z. Liang, and L. Yang, "Mobility-aware dependent task offloading in edge computing: A digital twin-assisted reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 24, no. 4, pp. 2979–2994, Apr. 2025.
[19] H. Li, C. Chen, H. Shan, P. Li, Y. C. Chang, and H. Song, "Deep deterministic policy gradient-based algorithm for computation offloading in IoV," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 3, pp. 2522–2533, Mar. 2024.
[20] P. Hou, X. Jiang, Z. Lu, B. Li, and Z. Wang, "Joint computation offloading and resource allocation based on deep reinforcement learning in C-V2X edge computing," *Appl. Intell.*, vol. 53, no. 19, pp. 22446–22466, 2023.
[21] Y. Guo et al., "Deep deterministic policy gradient-based intelligent task offloading for vehicular computing with priority experience playback," *IEEE Trans. Veh. Technol.*, vol. 73, no. 7, pp. 10655–10667, Jul. 2024.
[22] N. Fofana, A. B. Letaifa, and A. Rachedi, "Intelligent task offloading in vehicular networks: A deep reinforcement learning perspective," *IEEE Trans. Veh. Technol.*, vol. 74, no. 1, pp. 201–216, Jan. 2025.
[23] X. Fan, W. Gu, C. Long, C. Gu, and S. He, "Optimizing task offloading and resource allocation in vehicular edge computing based on heterogeneous cellular networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 5, pp. 7175–7187, May 2024.
[24] X. Chen, S. Hu, C. Yu, Z. Chen, and G. Min, "Real-time offloading for dependent and parallel tasks in cloud-edge environments using deep reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 3, pp. 391–404, Mar. 2024.
[25] Y. Li, X. Ge, B. Lei, X. Zhang, and W. Wang, "Joint task partitioning and parallel scheduling in device-assisted mobile edge networks," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14058–14075, Apr. 2024.
[26] S. Munawar, Z. Ali, M. Waqas, S. Tu, S. A. Hassan, and G. Abbas, "Cooperative computational offloading in mobile edge computing for vehicles: A model-based DNN approach," *IEEE Trans. Veh. Technol.*, vol. 72, no. 3, pp. 3376–3391, Mar. 2023.
[27] X. Xu, C. Yang, M. Bilal, W. Li, and H. Wang, "Computation offloading for energy and delay trade-offs with traffic flow prediction in edge computing-enabled IoV," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 15613–15623, Dec. 2023.
[28] Z. Xiao et al., "Multi-objective parallel task offloading and content caching in D2D-aided MEC networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 11, pp. 6599–6615, Nov. 2023.
[29] X. Wang et al., "Augmented Intelligence of Things for priority-aware task offloading in vehicular edge computing," *IEEE Internet Things J.*, vol. 11, no. 22, pp. 36002–36013, Nov. 2024.
[30] X. Zhu, Y. Luo, A. Liu, M. Z. A. Bhuiyan, and S. Zhang, "Multiagent deep reinforcement learning for vehicular computation offloading in IoT," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9763–9773, Jun. 2021.
[31] H. Shen, K. Zhang, M. Hong, and T. Chen, "Towards understanding asynchronous advantage actor-critic: Convergence and linear speedup," *IEEE Trans. Signal Process.*, vol. 71, pp. 2579–2594, 2023.
[32] Y. Ma, X. Zhu, S. Zhang, R. Yang, W. Wang, and D. Manocha, "TrafficPredict: Trajectory prediction for heterogeneous traffic-agents," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 6120–6127.
[33] L. Guo, J. Jia, J. Chen, and X. Wang, "QRMP-DQN empowered task offloading and resource allocation for the STAR-RIS assisted MEC systems," *IEEE Trans. Veh. Technol.*, vol. 74, no. 1, pp. 1252–1266, Jan. 2025.
[34] H. Zhai, X. Zhou, H. Zhang, and D. Yuan, "Delay minimization in hybrid edge computing networks: A DDQN-based task offloading approach," *IEEE Trans. Veh. Technol.*, vol. 73, no. 10, pp. 15098–15108, Oct. 2024.

**Jiaxin Du** received the PhD degree from the School of Software, Dalian University of Technology, China, in 2023. She is currently a lecturer with the College of Computer Science and Technology, Zhejiang University of Technology, China. Her current research interests include edge intelligence, security for wireless sensor networks and intelligent transportation systems.

**Jinfan Zhang** received the BE degree from Wenzhou University, Wenzhou, China, in 2023. He is currently working toward the master's degree in computer science and technology with the School of Computer Science, Zhejiang University of Technology, China. His research interests include edge computing and intelligent transportation systems.

**Guangjie Han** (Fellow, IEEE) received the PhD degree from Northeastern University, Shenyang, China, in 2004. He is currently a professor with the Department of Internet of Things Engineering, Hohai University, Changzhou, China. In February 2008, he finished his work as a Postdoctoral Researcher with the Department of Computer Science, Chonnam National University, Gwangju, Korea. From October 2010 to October 2011, he was a visiting research scholar with Osaka University, Suita, Japan. From January 2017 to February 2017, he was a visiting professor with City University of Hong Kong, China. From July 2017 to July 2020, he was a distinguished professor with Dalian University of Technology, China. His current research interests include Internet of Things, Industrial Internet, Machine Learning and Artificial Intelligence, Mobile Computing, Security and Privacy.

Dr. Han has more than 500 peer-reviewed journal and conference papers, in addition to 160 granted and pending patents. Currently, his H-index is 79 and i10-index is 365 in Google Citation (Google Scholar). The total citation count of his papers raises above 22000 times. Dr. Han is a fellow of the UK Institution of Engineering and Technology (FIET). He has served on the Editorial Boards of up to 10 international journals, including the *IEEE Transactions on Industrial Informatics, IEEE Transactions on Cognitive Communications and Networking, IEEE Transactions on Vehicular Technology, IEEE Transactions on Network and Service Management, IEEE Systems*, etc. He has guest-edited several special issues in *IEEE Journals and Magazines*, including the *IEEE Journal on Selected Areas in Communications, IEEE Communications, IEEE Wireless Communications, Computer Networks*, etc. Dr. Han has also served as chair of organizing and technical committees in many international conferences. He has been awarded 2020 IEEE Systems Journal Annual Best Paper Award and the 2017-2019 IEEE ACCESS Outstanding Associate Editor Award.

**Mengmeng Wang** received the PhD degree in control science and engineering from Zhejiang University, in 2024. She is currently an assistant professor with the College of Computer Science and Technology, Zhejiang University of Technology. Her research interests include image/video understanding, text-to-video/image-to-video generation, computer vision, robotics, and intelligent transportation systems.

**Guojiang Shen** received the BSc degree in control theory and control engineering, and the PhD degree in control science and engineering from Zhejiang University, Hangzhou, China, in 1999 and 2004, respectively. He is currently a professor with the College of Computer Science and Technology, Zhejiang University of Technology. His current research interests include artificial intelligence, Big Data analytics and intelligent transportation systems.

**Zhi Liu** received the BS degree in automatic control and the MS degree in system engineering from Xi'an Jiaotong University, Xi'an, China, in 1991 and 1994, respectively, and the PhD degree in computer science and technology from Zhejiang University, Hangzhou, China, in 2001. She is currently a professor with the College of Computer Science and Technology, Zhejiang University of Technology. She is a member of the China Computer Federation. Her current main research interests include intelligent transportation system and intelligent computing.

**Xiangjie Kong** (Senior Member, IEEE) received the BSc and PhD degrees from Zhejiang University, Hangzhou, China, in 2004 and 2009 respectively. He is currently a full professor with the College of Computer Science and Technology, Zhejiang University of Technology, China. Previously, he was an associate professor with the School of Software, Dalian University of Technology, China. He has published more than 200 scientific papers in international journals and conferences (with more than 170 indexed by ISI SCIE). His research interests include mobile computing, network science, and urban computing. He is a distinguished member of CCF and a member of ACM.